Einsatz von Large Language Models im Umfeld der Robotik hinsichtlich On-Premise-Lösungen

Moritz Jahn
Matrikelnummer: 3221169

15. März 2026

Inhaltsverzeichnis

Abkürzungsverzeichnis **v**

1 Einführung **1**
1.1 Problemstellung . . . 1
1.2 Stand der Technik . . . 1
1.3 Zielsetzung . . . 2
1.4 Aufbau der Arbeit . . . 2

2 Grundlagen Large Language Models **3**
2.1 Tokenisierung und Embedding . . . 3
2.2 Der Transformer-Block . . . 5
2.3 Ausgabeschicht, Training und Quantisierung . . . 8
2.4 Sprachverarbeitung: STT und TTS . . . 9
2.5 Das eingesetzte System . . . 11
2.6 Zusammenfassung . . . 13

3 On-Premise-Infrastruktur und Hardware **14**
3.1 Rechenoperationen eines LLMs – nano-GPT als Beispiel . . . 14
3.1.1 Modellkonfiguration . . . 15
3.1.2 Parameter-Aufschlüsselung . . . 15
3.1.3 Rechenoperationen im Forward Pass . . . 16
3.1.4 Skalierung auf LLaMA 7B . . . 17
3.2 Motivation für On-Premise-Betrieb . . . 17
3.2.1 Vergleich: Cloud vs. On-Premise . . . 18
3.3 Hardwareanforderungen . . . 18
3.3.1 GPU und VRAM . . . 18
3.3.2 CPU und Arbeitsspeicher . . . 19
3.3.3 Speichersystem . . . 19
3.4 Laboraufbau am IZPL . . . 19
3.5 Zusammenfassung . . . 20

4 Evaluation **21**
4.1 Methodik der Evaluation 21
4.2 Antwortzeiten und Performanz 22
4.3 Qualität der Konversation 23

5 Fazit und Ausblick **25**
5.1 Zusammenfassung der Ergebnisse 25
5.2 Ausblick auf die Masterarbeit 25

A Anhang **26**
A.1 Konfigurationsdateien 26
A.2 Testprotokolle 26
A.3 Zusätzliche Abbildungen 26

Literaturverzeichnis **27**

Abbildungsverzeichnis

2.1 Datenfluss eines Large Language Models 3
2.2 Datenfluss einer LLM-basierten Konversation: Spracheingabe (STT), Verarbeitung durch das Sprachmodell und Sprachausgabe (TTS). 4
2.3 Datenfluss einer LLM-basierten Konversation: Spracheingabe (STT), Verarbeitung durch das Sprachmodell und Sprachausgabe (TTS). 5
2.4 Datenfluss eines Large Language Models 10

4.1 Simulierter VRAM-Bedarf je Quantisierungsstufe im Vergleich zum 12-GB-Limit der RTX 3080 Ti 21
4.2 Generierungsgeschwindigkeit (Tokens/s) in Abhängigkeit der Quantisierung 22
4.3 Prefill-Latenz (Time-to-First-Token) über verschiedene Prompt-Längen (Q4_K_M) 22
4.4 Kumulative Token-Ausgabe über die Zeit bei der 4-Bit-Quantisierung . . . 23
4.5 Zeitlicher VRAM-Verlauf während der Modell-Ausführung gegliedert in Laden, Prefill und Generierung 23
4.6 Qualitätsverlust (Perplexity auf WikiText-2) durch Quantisierungsverfahren 24

Tabellenverzeichnis

2.1 VRAM-Bedarf eines 7B-Sprachmodells bei verschiedenen Quantisierungsstufen. 9
2.2 Architekturparameter – LLaMA 7B (fett) ist das in dieser Arbeit eingesetzte Modell. 12
2.3 Übersicht der eingesetzten Komponenten. 13

3.1 Konfiguration des nano-GPT-Modells [2]. 15
3.2 Parameter-Aufschlüsselung nano-GPT (85.584 Parameter). Die Output-Projektion teilt die Gewichte mit der Token-Embedding-Matrix (*weight tying*) [2]. 15
3.3 Geschätzte Rechenoperationen für nano-GPT bei $T = 6$ Eingabe-Tokens. Der Großteil entfällt auf die MLP-Schichten. 17
3.4 Vergleich Cloud-API vs. On-Premise-Betrieb. 18
3.5 Gängige GPUs für lokale LLM-Inferenz. ✓ : ausreichend, ✓✓ : gut geeignet, ✓✓✓ : optimal. 19
3.6 Hardware- und Software-Konfiguration am IZPL. 20

Abkürzungsverzeichnis

LLM Large Language Model

VLM Visual Language Model

NLP Natural Language Processing

GPU Graphics Processing Unit

CPU Central Processing Unit

VRAM Video Random Access Memory

RAM Random Access Memory

ROS Robot Operating System

API Application Programming Interface

TTS Text-to-Speech

STT Speech-to-Text

RAG Retrieval-Augmented Generation

GGUF GPT-Generated Unified Format

Kapitel 1

Einführung

Kaum ein anderes Thema hat sich seit dem Start 2021 so schnell verbreitet wie KI. Man hört in

1.1 Problemstellung

Mit den ständig wachsenden Ausbau von Datenzentrent weltweit um die Nachfrage nach Rechenleistung zu stillen wächst nicht nur die Nachfrage nach Hardware, sondern auch die Fragen nach Datenhoheit und Sicherheitsbedeken. Tech Giganten wie Google Anthropic oder XAi benötigen nicht nur für das Rennen nach den neusten und klügsten Modellen unvorstellbare Supercomputer. Alleine der Backlog von Hardwareprovidern wie Nvidia ist für das Jahr 2026 mit merhern Milliarden versehen. Denn um auf den aktuellen Markt der KI Modelle mithalten zu können ist es für die Firmen essenziell das beste Flagschiff modell im Sortiment zu haben. Der anfängliche Platzhirsch Openai mit Ihrem Modell Chatgpt (Chat Generative Pretrained Transformer) hat es anfang 2020 vorgemacht welches enorme öffentliche Interesse nach solchen Technologien besteht das alle anderen Unternehmen nachziehen mussten und langsam mit extremen geldmengen verwendet wurden um aufzuholen bzw. zu überholen. Wobei natürlich die Frage bleibt welche Risiken bieten die oft als Blackbox beschrieben Modellen hinsichtlich Datensicherheit. und gibt es alternativen die Vertgkeichbaret. Im folgenden wird die Chronolgie dieser schnell wachsenden Technologie Zusammengefasst.

1.2 Stand der Technik

Der Begriff maschinellen lernen ist nicht so neu in der Computerwelt schon seit anfang des 21 jahrhunderts gibts algorithmen die Spanennd wurde es ab 2017 wo der transformer zuerst von openai eingeführt wurde dies machte es möglich von seriell zu parallel und dies

mit einer GPU zu koppeln um viel größere datenmengen prallel zu computen.

Kaum ein anderes Thema hat sich seit dem Start 2021 so schnell verbreitet wie KI. Man hört in

1.3 Zielsetzung

Im Rahmen dieser Forschungsarbeit wird der Einsatz von On-Premise-LLMs, also vor Ort betriebenn, am Beispiel der Anwendungsmöglichkeit im Innovationszentrum für Produktion und Logistik (im folgenden IZPL) der OTH Regensburg untersucht. Dabei wird auf öffentlich verfügbare Modelle zurückgegriffen, die lokal auf einem Server des IZPL betrieben werden. Ziel ist es, die Grundlagen, technischen Anforderungen und Möglichkeiten solcher Systeme für künftige robotiknahe Anwendungen zu analysieren. Konkret dient diese wissenschaftliche Arbeit als Fundament für eine Abschlussarbeit im IZPL Regensburg.

1.4 Aufbau der Arbeit

Zunächst erfolgt eine Einführung in die mathematischen Konzepte, auf denen die Modelle basieren. Die im Folgenden aufgeführten Hardwarespezifikationen legen dar, welche Computing-Kapazitäten erforderlich sind, um sogenannte "künstliche Intelligenzen"(KI) zu erschaffen. Nach Evaluation einer Testanwendung in Kapitel 4 werden Sicherheits- und Fehlerbilder aufgezeigt. Die vorliegende Arbeit findet ihren Abschluss in Kapitel 6, das einen Ausblick auf zukünftige Forschungsgebiete gibt und ein Fazit zieht.

Kapitel 2

Grundlagen Large Language Models

Dieses Kapitel legt die theoretischen Grundlagen für das Verständnis moderner Large Language Models (LLMs). Es werden zunächst die mathematischen Grundlagen vom Embedding bis zur Softmax-Funktion erläutert und danach eine Text to speach Pipline vorgestellt. Das Kapitel beginnt mit Abbildung 2.1 welche sich auf den Grundlegenden Meachanismus eines LLMs fokussiert, den Transformer. LLMs haben wie in der Überschrift ersiochtlich aus sogenannten Prompts

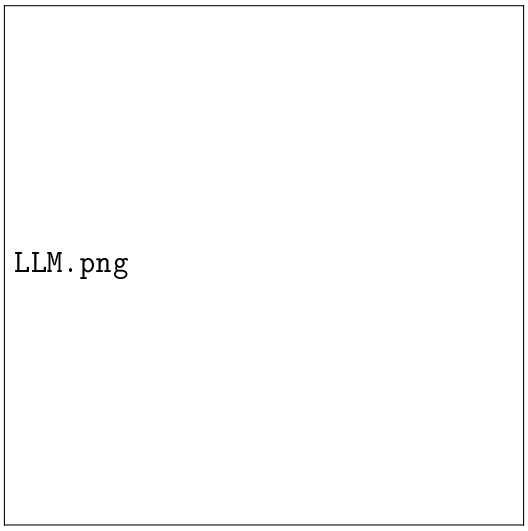

Abbildung 2.1: Datenfluss eines Large Language Models

2.1 Tokenisierung und Embedding

Bevor jedoch der Transformer zum Einsatz kommt, ist der erste Schritt eines jeden LLMs das sogenannte Tokenisieren. Ein Token im Kontext des maschinellen Lernens wurde im Januar dieses Jahres von Daniel Jurafsky & James H. Martin wie folgt definiert: ¨Tokens sind Einheiten, welche entstehen, nachdem wir Algorithmen wie BPE oder WordPiece anwenden, um Texteingaben für LLMs vorzubereiten.“[4] In Abbildung 2.1 wird dieser

Schritt mit dem ersten Block dargestellt und heißt übergeordnet Embedding. Beim Tokenisieren werden Wörter in Arrays umgewandelt. Zur Einordnung: Einer der ersten LLMs der Firma OpenAI, GPT-2, benutzt 768 Koordinaten, um den multidimensionalen Vektor eines einzigen Tokens aufzuspannen und um dessen Kontext zu speichern (siehe 2.1 in [1]). In Abbildung 2.2 ist das Vektor-Embedding für den Satz ¨LLMs with Igus Rebel 6 DOF“ mithilfe von Hauptachsentransformation (englisch Principal Component Analysis, kurz PCA) im dreidimensionalen Raum für GPT-2 dargestellt. Zur Tokenisierung wurde der bis heute standardmäßig verwendete Tokenisierungsalgorithmus Byte Pair Encoding verwendet.

Embedding.png

Abbildung 2.2: Datenfluss einer LLM-basierten Konversation: Spracheingabe (STT), Verarbeitung durch das Sprachmodell und Sprachausgabe (TTS).

Byte Pair Encoding (BPE) ist ein datengetriebener Teilwort-Tokenizer. Der Algorithmus beginnt mit einem Lookup-Tabel als Trainingskorpus und fasst iterativ das häufigste benachbarte Zeichenpaar,welche er im Trainigskorpuse nachschaut, zu einem neuen Token zusammen. Dieser Prozess wiederholt sich, bis eine gewünschte Vokabulargröße $|\mathcal{V}|$ erreicht ist [11]. Ursprünglich in den 90er Jahren für Speicherplatzreduzierung entwickelt zerlegt er seltene oder zusammengesetzte Wörter in häufige Teilwörter und kann somit prinzipiell jede Zeichenkette verarbeiten. Häufige Wörter erhalten dabei meist ein einzelnes Token, seltene Wörter oder Pluralformen werden in mehrere Tokens aufgeteilt.

Embedding2.png

Abbildung 2.3: Datenfluss einer LLM-basierten Konversation: Spracheingabe (STT), Verarbeitung durch das Sprachmodell und Sprachausgabe (TTS).

Statische Embeddings wie BPE haben jedoch ein grundlegendes Problem: Jedes Wort erhält genau einen festen Vektor, unabhängig vom Kontext. In Abbildung **??** ist dies am Wort Rebel dargestellt, das sowohl den igus-Leichtbauroboter als auch einen Aufständischen bezeichnen kann. Im Roboter-Kontext zeigt der Vektor in ähnliche Richtungen wie igus kuka und agilus. Im Kontext eines Rebellen hingegen in Richtung eines Verwandten wortes wie Outlaw. Diese Einordnungen hat das Modell während des Trainings gelernt. Welcher Kontext gemeint ist im Prompt kann ein statisches Embedding nicht Unterscheiden erst der im folgenden Abschnitt vorgestellte Transformer löst dieses Problem.

Token-Embedding:

$$x_i = e_i + p_i \tag{2.1}$$

2.2 Der Transformer-Block

Der Transformer-Block ist die zentrale Verarbeitungseinheit eines LLMs. Er besteht aus zwei Hauptoperationen zum ersten aus Multi-Head Self-Attention und einem Feed-Forward-Netzwerk (FFN) jeweils kombiniert mit einer Normierungsschicht und einer Residualverbindung:

$$x' = x + \text{Attention}\big(\text{Norm}(x)\big) \tag{2.2}$$

$$x'' = x' + \text{FFN}\big(\text{Norm}(x')\big) \tag{2.3}$$

Durch das N-fache Hintereinanderschalten solcher Blöcke entsteht eine tiefe Architektur, die schrittweise abstrakte Repräsentationen aufbaut.

Normierungsschicht. Zur Stabilisierung des Trainings tiefer Netze wird vor jeder Hauptoperation eine Normierungsschicht angewendet. Die klassische *Layer Normalization* berechnet Mittelwert μ und Standardabweichung σ über die Feature-Dimension:

$$\text{LayerNorm}(x) = \gamma \cdot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta, \quad \mu = \frac{1}{d}\sum_{i=1}^{d} x_i \tag{2.4}$$

Eine vereinfachte Variante ist die **RMSNorm** (Root Mean Square Normalization), die den Mittelwert nicht subtrahiert und damit rechnerisch effizienter ist:

$$\text{RMSNorm}(x) = \gamma \cdot \frac{x}{\sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \epsilon}} \tag{2.5}$$

Die Normierung verhindert, dass Aktivierungswerte über viele Schichten hinweg exponentiell wachsen oder verschwinden (Vanishing/Exploding Gradients).

Multi-Head Self-Attention. Der Self-Attention-Mechanismus ermöglicht es jedem Token, beim Aufbau seiner Repräsentation den gesamten bisherigen Kontext zu berücksichtigen. Drei lineare Projektionen erzeugen für jedes Token x_i die Vektoren Query q_i, Key k_i und Value v_i:

$$q_i = W_Q\, x_i, \quad k_i = W_K\, x_i, \quad v_i = W_V\, x_i \tag{2.6}$$

Die **Scaled Dot-Product Attention** berechnet Ähnlichkeitsgewichte zwischen Query und Keys und aggregiert die Values entsprechend:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right) V \tag{2.7}$$

Die **kausale Maske** M stellt sicher, dass Token t_i nur auf vorherige Positionen t_j mit $j \leq i$ zugreifen kann:

$$M_{ij} = \begin{cases} 0 & j \leq i \\ -\infty & j > i \end{cases} \tag{2.8}$$

Bei **Multi-Head Attention** werden h parallele Attention-Berechnungen in Unterräumen der Dimension $d_k = d/h$ ausgeführt und anschließend konkateniert:

$$\mathrm{MultiHead}(Q, K, V) = \left[\mathrm{head}_1 \parallel \cdots \parallel \mathrm{head}_h\right] W_O \tag{2.9}$$

Jeder Kopf kann dabei unterschiedliche Aspekte des Kontexts erlernen (z. B. syntaktische Abhängigkeiten, semantische Ähnlichkeit, Koreferenzen).

Feed-Forward-Netzwerk. Nach der Self-Attention verarbeitet ein positionsweises Feed-Forward-Netzwerk jede Token-Position unabhängig. In der ursprünglichen Transformer-Formulierung besteht es aus zwei linearen Transformationen mit einer nichtlinearen Aktivierung dazwischen [14]:

$$\mathrm{FFN}(x) = W_2 \cdot \mathrm{ReLU}(W_1\, x + b_1) + b_2 \tag{2.10}$$

Die innere Dimension d_{ff} ist typischerweise $4 \cdot d$. Das FFN speichert einen wesentlichen Teil des Faktenwissens des Modells.

Rotary Position Embeddings (RoPE). Neuere Architekturen verwenden funktionale Kodierungen, die die Positionsinformation direkt in den Attention-Mechanismus einbetten. **Rotary Position Embeddings (RoPE)** kodieren die Position durch eine Rotation des Query- und Key-Vektors mit einem positionsabhängigen Winkel [12]. Für einen zweidimensionalen Ausschnitt gilt:

$$\mathrm{RoPE}(x, i) = \begin{pmatrix} x_1 \cos(i\,\omega) - x_2 \sin(i\,\omega) \\ x_1 \sin(i\,\omega) + x_2 \cos(i\,\omega) \end{pmatrix}, \quad \omega_k = \frac{1}{10000^{2k/d}} \tag{2.11}$$

Da das Skalarprodukt zweier rotierter Vektoren nur von ihrer *relativen* Position abhängt, kodiert RoPE die Relativdistanz direkt im Attention-Mechanismus.

Residualverbindungen. Residualverbindungen (Skip Connections) addieren das Eingangssignal direkt zur Ausgabe einer Operation und lösen so das Problem der verschwindenden Gradienten in tiefen Netzen:

$$\mathrm{out} = f(x) + x \tag{2.12}$$

Jeder Transformer-Block besitzt zwei Residualverbindungen: eine nach der Self-Attention und eine nach dem FFN.

2.3 Ausgabeschicht, Training und Quantisierung

Nach dem letzten Transformer-Block wird die Ausgabe des letzten Tokens durch eine Normierung und eine lineare Projektion auf die Vokabulargröße abgebildet:

$$z = W_U \cdot \mathrm{Norm}\big(x_T^{(N)}\big), \quad z \in \mathbb{R}^{|\mathcal{V}|} \tag{2.13}$$

Der resultierende Vektor z (**Logits**) wird durch **Softmax** in eine Wahrscheinlichkeitsverteilung über das gesamte Vokabular umgewandelt:

$$P\big(t_{T+1} = w \mid t_1, \ldots, t_T\big) = \frac{\exp(z_w/\tau)}{\sum_j \exp(z_j/\tau)} \tag{2.14}$$

Der **Temperature**-Parameter τ steuert die Schärfe der Verteilung: kleine Werte ($\tau < 1$) machen das Modell deterministischer, große Werte ($\tau > 1$) erhöhen die Vielfalt. Das gewählte Token wird an die Sequenz angehängt und der Prozess wiederholt (**autoregressive Generierung**), bis ein Stopptoken erreicht ist. Gängige Sampling-Strategien sind **Top-k Sampling** (nur die k wahrscheinlichsten Tokens) und **Top-p (Nucleus) Sampling** (kleinste Menge mit kumulativer Wahrscheinlichkeit $\geq p$).

Pretraining. Beim Pretraining lernt das Modell, das jeweils nächste Token einer Text-Sequenz vorherzusagen (**Causal Language Modeling, CLM**). Die Verlustfunktion ist die mittlere negative Log-Likelihood (Cross-Entropy) über alle Positionen:

$$\mathcal{L}_{\mathrm{CLM}}(\theta) = -\frac{1}{T} \sum_{t=1}^{T} \log P(w_t \mid w_1, \ldots, w_{t-1}; \theta) \tag{2.15}$$

Die Gewichte θ werden durch stochastischen Gradientenabstieg (Adam-Optimizer) minimiert. Das Pretraining auf großen Textkorpora befähigt das Modell, allgemeines Weltwissen und sprachliche Strukturen zu erlernen.

Finetuning und RLHF. Nach dem Pretraining ist ein Modell nur auf Token-Vervollständigung optimiert. Durch **Supervised Fine-Tuning (SFT)** auf Paaren aus Anweisung und gewünschter Antwort wird das Modell auf Konversationen spezialisiert. **Reinforcement Learning from Human Feedback (RLHF)** verfeinert das Modell weiter: Menschliche Bewerter vergleichen Modellantworten, daraus wird ein *Reward Model* trainiert, und das Sprachmodell wird so optimiert, dass sein Reward maximiert wird.

Quantisierung. Modellgewichte werden standardmäßig als 16-Bit-Gleitkommazahlen gespeichert. Auf lokaler Hardware mit begrenztem VRAM ist dies für große Modelle oft nicht praktikabel. **Quantisierung** reduziert die Bitbreite der Gewichte, um Speicherbedarf und

Inferenzgeschwindigkeit zu verbessern:

$$\mathrm{VRAM}_{\mathrm{min}} \approx \frac{N_{\mathrm{Parameter}} \times \mathrm{Bitbreite}}{8 \times 10^9} \; [\mathrm{GB}] \tag{2.16}$$

Format	Bitbreite	VRAM (7B-Modell)	Qualitätsverlust
FP16	16 Bit	≈14 GB	Referenz
INT8	8 Bit	≈7 GB	minimal
Q4_K_M	4 Bit	≈4 GB	gering, praxistauglich
Q2_K	2 Bit	≈2 GB	deutlich spürbar

Tabelle 2.1: VRAM-Bedarf eines 7B-Sprachmodells bei verschiedenen Quantisierungsstufen.

Das **GGUF**-Format hat sich als portables Standardformat für quantisierte Gewichte etabliert und wird von `llama.cpp` und `ollama` nativ unterstützt.

2.4 Sprachverarbeitung: STT und TTS

Da LLMs ausschließlich Text verarbeiten können, muss gesprochene Sprache zunächst transkribiert werden. Die Umwandlung von Audio in Text bezeichnet man als **Automatic Speech Recognition (ASR)** oder **Speech-to-Text (STT)**.

Die Vibration der Stimmlippen versetzt Luftmoleküle in Schwingung. Durch die Kompressibilität der Luft entsteht eine sich ausbreitende Schallwelle, die physikalisch der Schwankung des Luftdrucks (Schalldruck p, Einheit: Pa) um den atmosphärischen Ruhedruck (1013,25 hPa) entspricht [7]. Subjektiv wird Schall durch *Klangfarbe* und *Lautstärke* beschrieben; die physikalischen Äquivalente sind die Frequenz f und die Amplitude des Schalldrucks. Die Frequenz berechnet sich über die Periodendauer T:

$$f = \frac{1}{T} \tag{2.17}$$

Ein Signal mit genau einer Frequenzkomponente heißt *Ton* (z. B. der Kammerton A4 bei 440 Hz), eine Überlagerung ganzzahlig verwandter Frequenzen *Klang*. Menschliche Sprache ist eine Überlagerung harmonischer Schwingungen im Bereich von 80 Hz bis 12 kHz [8]. Da die menschliche Lautstärkewahrnehmung logarithmisch ist, wird der *Schalldruckpegel* L eingeführt:

$$L = 20 \cdot \lg\left(\frac{p}{p_0}\right) \quad \text{in dB} \tag{2.18}$$

wobei $p_0 = 2 \cdot 10^{-5}$ Pa der Hörschwelle bei 1 kHz (0 dB) entspricht. Die Schmerzgren-

ze liegt bei 200 Pa bzw. 140 dB. Die Angabe dB ist keine Maßeinheit, sondern weist auf das logarithmische Bildungsgesetz hin; 1 dB entspricht etwa der menschlichen Unterschiedsschwelle zwischen zwei Lautstärken. Die wahrgenommene Lautstärke (Einheit: phon) hängt zusätzlich von der Frequenz ab und wurde empirisch ermittelt, wobei die Hörschwelle bei 0 phon und die Schmerzgrenze bei 120 phon definiert ist.

Embedding.png

Abbildung 2.4: Datenfluss eines Large Language Models

Das Eingangssignal ist eine Audiodatei (typischerweise 16 kHz Mono). Um es für ein neuronales Netz nutzbar zu machen, wird es in ein **Log-Mel-Spektrogramm** umgewandelt – eine Zeit-Frequenz-Darstellung, die der Wahrnehmung des menschlichen Gehörs nachgebildet ist:

$$S_{\text{mel}}(m,t) = \log\left(\sum_k |X(k,t)|^2 \cdot H_m(k) + \epsilon\right) \tag{2.19}$$

wobei $X(k,t)$ das Kurzzeit-Fourierspektrum, $H_m(k)$ die m-te Mel-Filterbank und ϵ ein Glättungsterm ist. Ein neuronales Netz – typischerweise ein Transformer (Encoder-Decoder) – erzeugt aus diesem Spektrogramm direkt die Texttranskription.

Text-to-Speech (TTS) bezeichnet die Synthese von Sprache aus Text. Moderne neuro-

nale TTS-Systeme erzeugen natürlich klingende Sprache in einem mehrstufigen Prozess:

1. **Phonemisierung:** Der Eingabetext wird normalisiert und in eine phonetische Repräsentation (Phoneme) umgewandelt.
2. **Akustisches Modell:** Aus der Phonemsequenz wird ein Mel-Spektrogramm generiert – eine Zeit-Frequenz-Darstellung, die Tonhöhe, Geschwindigkeit und Klangfarbe kodiert.
3. **Vocoder:** Das Mel-Spektrogramm wird in ein Audiosignal (Wellenform) umgewandelt.

End-to-End-Modelle wie **VITS** (Variational Inference with adversarial learning for end-to-end TTS) [6] kombinieren diese Stufen in einem einzigen Modell und werden direkt von Text zu Audio trainiert. Die Trainingsschleife minimiert eine Kombination aus Rekonstruktionsfehler und Adversarial Loss:

$$\mathcal{L}_{\text{VITS}} = \mathcal{L}_{\text{recon}} + \lambda \, \mathcal{L}_{\text{adv}} \tag{2.20}$$

2.5 Das eingesetzte System

Auf Basis der in den vorherigen Abschnitten beschriebenen Theorie wird in dieser Arbeit eine vollständige Sprachpipeline eingesetzt, die ausschließlich lokal ohne Cloud-Anbindung betrieben wird. Die drei Kernkomponenten sind:

Spracherkennung: Whisper. Whisper [10] ist ein von OpenAI veröffentlichtes Open-Source-Modell zur automatischen Spracherkennung. Es wurde auf 680.000 Stunden mehrsprachiger Audiodaten trainiert und basiert auf einer Transformer-Encoder-Decoder-Architektur. Der Encoder verarbeitet 80-Kanal Log-Mel-Spektrogramme (vgl. Gleichung 2.19), der Decoder generiert autoregressiv die Textausgabe. Whisper unterstützt mehrsprachige Erkennung ohne sprachspezifische Anpassung und ist robust gegenüber Umgebungsgeräuschen. In dieser Arbeit wird Whisper lokal auf der IZPL-Hardware ausgeführt. Ein gesprochener Satz wie „Wie hoch ist die aktuelle Raumtemperatur?“ wird innerhalb weniger hundert Millisekunden transkribiert und als Zeichenkette an das Sprachmodell übergeben.

Sprachmodell: LLaMA 7B. LLaMA 7B [13] ist ein von Meta entwickeltes Open-Source-Sprachmodell mit 6,7 Milliarden Parametern. Es verwendet die in Abschnitt 2.2 beschriebene Transformer-Architektur mit folgenden Spezifikationen:

Modell	N	d	h	d_{ff}	Parameter
GPT-2 (small)	12	768	12	3.072	124 M
LLaMA 7B	**32**	**4.096**	**32**	**11.008**	**6,7 B**
LLaMA 13B	40	5.120	40	13.824	13 B
GPT-3	96	12.288	96	49.152	175 B

Tabelle 2.2: Architekturparameter – LLaMA 7B (fett) ist das in dieser Arbeit eingesetzte Modell.

Im Vergleich zum Standard-Transformer setzt LLaMA 7B folgende Anpassungen ein:

- **RMSNorm** (statt LayerNorm) vor jeder Operation (vgl. Gleichung 2.5)
- **SwiGLU**-Aktivierung im FFN (statt ReLU):

$$\text{FFN}_{\text{SwiGLU}}(x) = \big(\text{Swish}(xW_1) \odot (xW_3)\big)\, W_2, \quad \text{Swish}(z) = z \cdot \sigma(z) \tag{2.21}$$

- **RoPE** Positionskodierung (vgl. Gleichung 2.11)
- Maximale Kontextlänge: $T_{\max} = 4096$ Tokens

Das Modell wird im **Q4_K_M**-Format über `ollama` lokal betrieben, was einen VRAM-Bedarf von ca. 4 GB ergibt und den Betrieb auf der verfügbaren Hardware ohne relevante Qualitätseinbußen ermöglicht.

Sprachsynthese: Piper TTS mit eigenem Stimmmodell. Piper TTS [3] ist ein schnelles, Open-Source-TTS-System, das auf der VITS-Architektur (vgl. Abschnitt 2.4) basiert und für den lokalen, ressourcenschonenden Betrieb optimiert ist. In dieser Arbeit wurde ein eigenes Stimmmodell trainiert, um die Sprachausgabe natürlicher zu gestalten. Das Vorgehen folgt dem Verfahren von NetworkChuck [9]:

1. **Datenaufbereitung:** Aufnahme von Sprachsamples mit dem Piper Recording Studio. Die Samples werden rauschbereinigt, auf 22.050 Hz normalisiert und den zugehörigen Transkripten zugeordnet.
2. **Finetuning:** Training ausgehend von einem vortrainierten deutschen Piper-Checkpoint über 6000 Epochen:

```
python3 -m piper_train \
  --dataset-dir ~/sprachdaten \
  --accelerator gpu --gpus 1 \
  --batch-size 32 --max_epochs 6000 \
  --resume_from_checkpoint base.ckpt \
  --precision 16 --quality medium
```

3. **ONNX-Export:** Das trainierte Modell wird in das ONNX-Format (Open Neural Network Exchange) exportiert, das eine hardwareunabhängige Inferenz ohne GPU ermöglicht:

```
python3 -m piper_train.export_onnx \
  checkpoint/epoch=5999.ckpt \
  output/stimme.onnx
```

Das resultierende Modell (`stimme.onnx`, ca. 60 MB) wird zusammen mit der Konfigurationsdatei `stimme.onnx.json` in Piper eingebunden und ermöglicht die Echtzeitsynthese auf der lokalen Hardware.

2.6 Zusammenfassung

Dieses Kapitel hat zunächst die theoretischen Grundlagen der LLM-Architektur – von der Tokenisierung über den Transformer-Block bis zur Softmax-Ausgabe – erläutert. Anschließend wurde das in dieser Arbeit eingesetzte System beschrieben:

Komponente	Eingesetztes Modell	Besonderheit
STT	Whisper	lokal, mehrsprachig, 80 Mel-Kanäle
LLM	LLaMA 7B (Q4_K_M, GGUF)	N=32, d=4096, ≈4 GB VRAM
TTS	Piper TTS + eigenes ONNX-Modell	VITS, f_s=22.050 Hz, kein Cloud

Tabelle 2.3: Übersicht der eingesetzten Komponenten.

Kapitel 3

On-Premise-Infrastruktur und Hardware

Dieses Kapitel beschreibt die Hardwareanforderungen für den lokalen Betrieb eines Large Language Models und den konkreten Laboraufbau am IZPL. Als Einstieg wird zunächst anhand eines minimalen Beispielmodells (nano-GPT, 85.584 Parameter) gezeigt, welche Rechenoperationen ein forward pass tatsächlich erfordert – bevor diese Erkenntnisse auf das in dieser Arbeit eingesetzte LLaMA 7B-Modell skaliert werden. Der Grund für den Start des Aufschwungs des LLMs in den 2020er Jahren ist vor allem auf die Erfindung des Transformers zurückzuführen dieser Algorithmus machte es möglich anstatt wie vorher nur möglich eine operation gleichzeitig auszufühen konnten nun tausende parallel ausgeführt werden.

3.1 Rechenoperationen eines LLMs – nano-GPT als Beispiel

Wie in Kapitel **??** bereits angeschnitten sind Large language models im ersten Schritt mit Embedding layers ausgestattet. Um die Skalierung allein dieses schrittes zu veranaschaulichen wie bereits genannt hat Gpt2 768 dimensionale Vektoren als Start vor jeder Matrixmultiplikation GPT3 jedoch hat für das 175 Milliarden Parameter modell 12288 dimensionen. **??** Um zu verstehen, warum LLMs hohe Hardware-Anforderungen stellen, ist es hilfreich, die Rechenoperationen eines konkreten, kleinen Modells vollständig nachzuvollziehen. Brendan Bycroft stellt mit der interaktiven Visualisierung [2] ein nano-GPT-Modell mit **85.584 Parametern** zur Verfügung, das schrittweise — von der Tokenisierung bis zum Softmax-Ausgang — alle Operationen zeigt.

3.1.1 Modellkonfiguration

Das nano-GPT-Modell ist darauf ausgelegt, eine Buchstabensequenz zu sortieren (z. B. `C B A B B C` → `A B B B C C`). Die Modellparameter sind:

Parameter	Bezeichnung	Wert
$\lvert\mathcal{V}\rvert$	Vokabulargröße (Zeichen)	65
T	Kontextlänge (Tokens)	64
C	Embedding-Dimension	48
L	Anzahl Transformer-Blöcke	3
H	Anzahl Attention-Heads	3
C/H	Head-Dimension	16
$4C$	MLP innere Dimension	192

Tabelle 3.1: Konfiguration des nano-GPT-Modells [2].

3.1.2 Parameter-Aufschlüsselung

Die 85.584 Parameter verteilen sich wie folgt auf die einzelnen Schichten:

Schicht	Formel	Parameter
Token-Embedding-Matrix	$\lvert\mathcal{V}\rvert \times C = 65 \times 48$	3.120
Positions-Embedding	$T \times C = 64 \times 48$	3.072
Je Transformer-Block (×3):		
LayerNorm 1 (γ, β)	$2 \times C$	96
Q, K, V Projektion	$3 \times (C \times C + C) = 3 \times 2.352$	7.056
Attention Output	$C \times C + C$	2.352
LayerNorm 2	$2 \times C$	96
MLP Up (FC1)	$C \times 4C + 4C = 48 \times 192 + 192$	9.408
MLP Down (FC2)	$4C \times C + C = 192 \times 48 + 48$	9.264
Summe pro Block		28.272
Summe 3 Blöcke		84.816
Finale LayerNorm	$2 \times C$	96
Output-Projektion	$C \times \lvert\mathcal{V}\rvert$ – weight tying	0
Gesamt		**85.584**

Tabelle 3.2: Parameter-Aufschlüsselung nano-GPT (85.584 Parameter). Die Output-Projektion teilt die Gewichte mit der Token-Embedding-Matrix (*weight tying*) [2].

3.1.3 Rechenoperationen im Forward Pass

Die Visualisierung [2] zeigt jede Addition und Multiplikation im forward pass. Die dominierenden Operationen sind Matrix-Multiplikationen. Für eine Eingabesequenz der Länge $T = 6$ (das Sortierbeispiel) ergeben sich folgende charakteristische Operationen:

Token- und Positions-Embedding. Für T Tokens werden T Zeilen aus der Embedding-Matrix ausgelesen ($T \times C = 6 \times 48 = 288$ Werte) und anschließend elementweise mit den Positionsvektoren addiert (288 Additionen). Dies ist reine Lookup-/Additionsarbeit – kaum Rechenlast.

Q, K, V Projektionen (pro Block). Für jedes der T Tokens werden Q, K und V berechnet:

$$\text{FLOPs} = 3 \times T \times C^2 \times 2 = 3 \times 6 \times 48^2 \times 2 = 82.944 \tag{3.1}$$

(Faktor 2 für Multiply-Add-Fusion). Bei $L = 3$ Blöcken ergibt das $3 \times 82.944 = 248.832$ FLOPs allein für die Projektionen.

Attention-Score-Matrix. Die Attention-Score-Matrix hat die Form $(T \times T)$ pro Head. Für $T = 6$, $H = 3$ Köpfe und Head-Dimension $d_k = 16$:

$$\text{FLOPs}_{\text{scores}} = H \times T^2 \times d_k \times 2 = 3 \times 36 \times 16 \times 2 = 3.456 \tag{3.2}$$

Die anschließende Softmax-Berechnung erfolgt zeilenweise über T Elemente pro Head (Exponentialfunktion + Summation + Division).

MLP (pro Block). Das zweilagige Feed-Forward-Netz verarbeitet jede der T Positionen unabhängig:

$$\text{FLOPs}_{\text{MLP}} = T \times (C \times 4C + 4C \times C) \times 2 = 6 \times (48 \cdot 192 + 192 \cdot 48) \times 2 = 442.368 \tag{3.3}$$

Operation	FLOPs
Embedding (Lookup + Add)	< 1.000
Q/K/V-Projektionen (3 Blöcke)	248.832
Attention-Scores (3 Blöcke)	10.368
Attention-Weighted Sum	10.368
Output-Projektion Attention	110.592
MLP FC1 + FC2 (3 Blöcke)	1.327.104
LayerNorms + Residuals	~ 10.000
Output-Projektion + Softmax	~ 37.000
Gesamt (geschätzt)	$\approx$ **1,75** MFLOPs

Tabelle 3.3: Geschätzte Rechenoperationen für nano-GPT bei $T = 6$ Eingabe-Tokens. Der Großteil entfällt auf die MLP-Schichten.

Gesamtoperationen (nano-GPT, $T = 6$).

Wichtige Erkenntnis. Das MLP-Netzwerk dominiert den Rechenaufwand – weit vor der Attention-Berechnung. Bei einer Sequenzlänge von $T = 6$ entfällt auf die drei MLP-Blöcke allein $\approx 76\,\%$ der FLOPs. Die Attention-Scores wachsen mit $\mathcal{O}(T^2)$, werden bei kurzen Sequenzen aber von den linearen $\mathcal{O}(C^2)$-Matrixmultiplikationen dominiert.

3.1.4 Skalierung auf LLaMA 7B

Dieselbe Struktur gilt für LLaMA 7B, jedoch mit deutlich größeren Dimensionen ($C = 4096$, $L = 32$, $H = 32$, $T_{\max} = 4096$). Die Anzahl der Floating-Point-Operationen pro Token lässt sich grob abschätzen als:

$$\text{FLOPs/Token} \approx 2 \times N_{\text{Parameter}} = 2 \times 6{,}7 \times 10^9 \approx 13{,}4 \text{ GFLOPs} \tag{3.4}$$

Diese Faustformel [5] gilt für Autoregressive Inferenz, bei der $T \ll C$ gilt. Bei einer Antwort mit 200 Token ergibt das:

$$200 \times 13{,}4 \text{ GFLOPs} = 2{,}68 \text{ TFLOPs} \tag{3.5}$$

3.2 Motivation für On-Premise-Betrieb

Trotz der verfügbaren Cloud-LLM-Dienste (OpenAI API, Anthropic, Gemini) gibt es gewichtige Gründe für einen lokalen Betrieb:

- **Datenschutz und Compliance:** Unternehmensinterne Daten, Sensordaten und

Strategieinformationen dürfen gemäß DSGVO und internen Richtlinien in der Regel nicht an externe Server übermittelt werden. Bei einem On-Premise-System verlassen die Daten das lokale Netzwerk nicht.

- **Verfügbarkeit und Latenz:** Cloud-Dienste sind von Internetzugang und der Verfügbarkeit des Anbieters abhängig. Lokale Modelle können in Echtzeit und ohne Netzwerkverzögerung reagieren – relevant für robotische Anwendungen.
- **Kosten:** Bei dauerhaftem, hochfrequentem Einsatz übersteigen API-Kosten schnell die Investitionskosten eigener Hardware.
- **Anpassbarkeit:** Lokale Modelle können feinabgestimmt, quantisiert und mit systemspezifischem Kontext (z. B. RAG) versehen werden, ohne von Anbieterbeschränkungen abhängig zu sein.

3.2.1 Vergleich: Cloud vs. On-Premise

Kriterium	Cloud (API)	On-Premise
Datenschutz	Daten verlassen das Netz	vollständig lokal
Latenz	100–2000 ms (Netz + Server)	50–500 ms (lokal)
Verfügbarkeit	abhängig von Anbieter	lokal kontrollierbar
Kosten	pro Token (variabel)	Einmalinvestition
Wartung	keine	selbstverantwortlich
Modellwahl	begrenzt (Anbieter-API)	frei wählbar

Tabelle 3.4: Vergleich Cloud-API vs. On-Premise-Betrieb.

3.3 Hardwareanforderungen

Aus den in Abschnitt 3.1 beschriebenen Rechenoperationen und der Skalierung auf LLaMA 7B ergeben sich konkrete Hardware-Anforderungen.

3.3.1 GPU und VRAM

Die GPU ist die kritische Ressource für LLM-Inferenz. Zwei Faktoren sind maßgebend:

1. **VRAM:** Die gesamten Modellgewichte müssen im GPU-Speicher liegen. Für LLaMA 7B im Format Q4_K_M sind das ca. 4–5 GB (vgl. Tabelle 2.1). Zusätzlich werden die KV-Cache-Aktivierungen für den Kontext benötigt:

$$\text{KV-Cache} \approx 2 \times L \times T \times C \times \text{Bytes/Wert} \tag{3.6}$$

Für LLaMA 7B bei $T = 2048$, $L = 32$, $C = 4096$ und FP16: $2 \times 32 \times 2048 \times 4096 \times 2 \approx 1{,}07\,\text{GB}$.

2. **Speicherbandbreite:** Bei der Autoregressive Inferenz wird pro Token die gesamte Gewichtsmatrix einmal gelesen – Memory-Bound-Betrieb. Eine hohe Speicherbandbreite (GB/s) ist entscheidender als rohe FLOP-Leistung.

GPU	VRAM	Bandbreite	LLaMA 7B Q4
NVIDIA RTX 3060	12 GB	360 GB/s	✓
NVIDIA RTX 3090	24 GB	936 GB/s	✓✓
NVIDIA RTX 4090	24 GB	1.008 GB/s	✓✓
AMD RX 7900 XTX	24 GB	960 GB/s	✓✓
NVIDIA A100 (80 GB)	80 GB	2.000 GB/s	✓✓✓

Tabelle 3.5: Gängige GPUs für lokale LLM-Inferenz. ✓ : ausreichend, ✓✓ : gut geeignet, ✓✓✓ : optimal.

3.3.2 CPU und Arbeitsspeicher

Falls kein ausreichender VRAM vorhanden ist, können Modellschichten auf den CPU-RAM ausgelagert werden (**CPU Offloading**). Dies senkt die VRAM-Anforderungen drastisch, reduziert aber den Durchsatz deutlich (typisch 1–3 Token/s statt 10–30 Token/s).

Mindestanforderungen für LLaMA 7B Q4 mit CPU-Offloading:

- **RAM:** $\geq 16\,\text{GB}$ (Modell: $\approx 4\,\text{GB}$ + Betriebssystem + KV-Cache)
- **CPU:** Multi-Core mit hoher Speicherbandbreite (z. B. AMD Ryzen 9 oder Intel Core i9)

3.3.3 Speichersystem

Das Laden eines GGUF-Modells (ca. 4 GB) vom Speichermedium beeinflusst die Startlatenz. Mit einer NVMe-SSD (3.000–7.000 MB/s) beträgt die Ladezeit $\approx$ 1–2 Sekunden, mit einer HDDs (100–200 MB/s) $\approx$ 20–40 Sekunden.

3.4 Laboraufbau am IZPL

Der konkrete Hardware-Aufbau für diese Arbeit befindet sich im Labor des **IZPL** (Innovationszentrum für Produktionsund Logistiklösungen) der OTH Regensburg. Das System umfasst:

Komponente	Spezifikation
Prozessor (CPU)	
Arbeitsspeicher	
Grafikkarte (GPU)	
Speicher	
Betriebssystem	
LLM-Laufzeitumgebung	`ollama`
Modell	LLaMA 7B (Q4_K_M, GGUF)
STT	Whisper (lokal)
TTS	Piper TTS (ONNX, lokal)

Tabelle 3.6: Hardware- und Software-Konfiguration am IZPL.

Mit dieser Konfiguration werden folgende Betriebswerte erreicht:

- **Inferenzgeschwindigkeit:**
- **Latenz (Erstantwort):**
- **VRAM-Auslastung:**

3.5 Zusammenfassung

Dieses Kapitel hat den Rechenbedarf von LLMs anhand des nano-GPT-Modells (85.584 Parameter, [2]) von Grund auf hergeleitet. Der forward pass eines nano-GPT mit $T = 6$ Token erfordert $\approx 1{,}75$ MFLOPs; skaliert auf LLaMA 7B mit 200 Ausgabe-Token sind es $\approx 2{,}68$ TFLOPs.

Die kritischen Hardware-Ressourcen für lokale LLM-Inferenz sind **VRAM** (Modellgewichte + KV-Cache) und **Speicherbandbreite** (Memory-Bound-Betrieb). LLaMA 7B im Q4_K_M-Format kann mit einer modernen Consumer-GPU (≥ 6 GB VRAM) lokal betrieben werden. Die IZPL-Hardware erfüllt diese Anforderungen und ermöglicht den On-Premise-Betrieb der vollständigen Sprachpipeline (Whisper + LLaMA 7B + Piper TTS) ohne Cloud-Anbindung.

Kapitel 4

Evaluation

Diese Kapitel führt eine Versuchsreihe durch welche mit dem im IZPL vorhanden Hardwaresetup durhc. Als LLM wurde 7Milliarden Paramter mit Piper als Text to Speach Modell. Das Githun Repository **??** dient hierbei als Startgerüst und wurde nur bzgl. geringen Änderungen verändert.

4.1 Methodik der Evaluation

Es werden die Durchschnittliche Antwortzeit die WER und die Qualität der Antwort verglichen. Zur Beurteilung der Hardwareanforderungen wird zudem der VRAM-Bedarf betrachtet.

abbildungen/llama7b_vram.jpg

Abbildung 4.1: Simulierter VRAM-Bedarf je Quantisierungsstufe im Vergleich zum 12-GB-Limit der RTX 3080 Ti

4.2 Antwortzeiten und Performanz

Die Performanz des Modells wird primär über die Token-Generierungsgeschwindigkeit sowie die initiale Latenz (Time-to-First-Token) gemessen.

abbildungen/llama7b_tokens_per_s.jpg

Abbildung 4.2: Generierungsgeschwindigkeit (Tokens/s) in Abhängigkeit der Quantisierung

abbildungen/llama7b_ttft.jpg

Abbildung 4.3: Prefill-Latenz (Time-to-First-Token) über verschiedene Prompt-Längen (Q4_K_M)

Ein detaillierter Blick auf den Generierungsprozess im zeitlichen Verlauf zeigt die kontinuierliche Token-Ausgabe sowie die dynamische VRAM-Auslastung während der Inferenz.

abbildungen/llama7b_token_gen.jpg

Abbildung 4.4: Kumulative Token-Ausgabe über die Zeit bei der 4-Bit-Quantisierung

abbildungen/llama7b_vram_trace.jpg

Abbildung 4.5: Zeitlicher VRAM-Verlauf während der Modell-Ausführung gegliedert in Laden, Prefill und Generierung

4.3 Qualität der Konversation

Für LLMs gibt es viele Benchmarks zu den aus objektiver sicht zählenden gehört die sogenannte Perplexity. Durch die Quantisierung des Modells lässt sich eine minimale Einbuße bei der Sprachqualität beobachten, welche jedoch bei 4-Bit-Gewichten noch als praxistauglich gilt.

abbildungen/llama7b_perplexity.jpg

Abbildung 4.6: Qualitätsverlust (Perplexity auf WikiText-2) durch Quantisierungsverfahren

Kapitel 5

Fazit und Ausblick

Zusammenfassend ist zu sagen dass On premise lösungen welche lokal auf server betrieben werden in bestimmmten anwendungsfällen eine sehr gut alternative zu cloud basierten lösungen bieten. Robotikanwendungen in innerbetrieblichen Bereich sind oftmals überschaubar genung bezogen auf veränderlichen variabeln, dass die anpassung der hyperparamter eines klein bis mittlgroßen modells überschaubar sind und demnach gut lokal betrieben werden können. Für große Modelle wie die meißsten sie heute kennen sind große datenzentren nötig mit hohen energiekosten welche für einzelen anwendungen nur wenig sinn ergeben. Dennoch ist immer abzugwwägen ob es in Zukunft für ein wachsendes Unternehmen nicht von Vorteil wäre geld in die hand zu nehmen um eine Serverstruktur zu schaffen welche sich vom Cloud geschäft ein stück weit loslöst und eigen LLM auf eigenen Servern betreibt. Zudem wird trotz wachsender Compute möglichkeiten eine linie nicht überschritten werden können laut **??**

5.1 Zusammenfassung der Ergebnisse

5.2 Ausblick auf die Masterarbeit

Die Masterarbeit knüpft an vielen der hier erarbeiteten Grundlagen direkt an und umfasst weiterfürhende Themen wie vision language action models in der Robotik. Zusammene mit einer weiteren Forschungs und Entwicklungsarbeit zum Thema ROS2 auf einem IGus Rebel 6 DOf wird ein Testaufbau entwickelt welcher konkret Robotik und Large Languange models im Kontext von Open source und on premise evaluiert. Generell bleit abzuwarten ob der hype um diese neue Technologie den erhofften wandel der gesellschaft standhalten kann oder doch kompromisse eingegangen werden müssen. auf jden fall wurde durch diese Forschungsarbeit ein Grundstein zur Erklärung der Modelle gelegt.

Anhang A

Anhang

A.1 Konfigurationsdateien

A.2 Testprotokolle

A.3 Zusätzliche Abbildungen

Literaturverzeichnis

[1] Tom B. Brown u. a. *Language Models are Few-Shot Learners*. 2020. arXiv: 2005.14165 [cs.CL]. URL: https://arxiv.org/abs/2005.14165.

[2] Brendan Bycroft. *LLM Visualization*. 2023. URL: https://bbycroft.net/llm.

[3] Michael Hanson. *Piper: A fast, local neural text to speech system*. 2023. URL: https://github.com/rhasspy/piper.

[4] Daniel Jurafsky und James H. Martin. *Speech and Language Processing*. 3rd. 2026. URL: https://web.stanford.edu/~jurafsky/slp3/.

[5] Jared Kaplan u. a. *Scaling Laws for Neural Language Models*. 2020. arXiv: 2001.08361. URL: https://arxiv.org/abs/2001.08361.

[6] Jaehyeon Kim, Jungil Kong und Juhee Son. *Conditional Variational Autoencoder with Adversarial Learning for End-to-End Text-to-Speech*. 2021. arXiv: 2106.06103. URL: https://arxiv.org/abs/2106.06103.

[7] Michael Möser. *Technische Akustik*. Berlin, Heidelberg: Springer Vieweg, 2015.

[8] Gerhard Müller und Michael Möser. *Handbuch der Technischen Akustik*. Berlin, Heidelberg: Springer Vieweg, 2015.

[9] NetworkChuck. *Clone Your Own Voice with AI*. 2024. URL: https://www.youtube.com/watch?v=placeholder.

[10] Alec Radford u. a. *Robust Speech Recognition via Large-Scale Weak Supervision*. 2022. arXiv: 2212.04356. URL: https://arxiv.org/abs/2212.04356.

[11] Rico Sennrich, Barry Haddow und Alexandra Birch. *Neural Machine Translation of Rare Words with Subword Units*. 2016. arXiv: 1508.07909. URL: https://arxiv.org/abs/1508.07909.

[12] Jianlin Su u. a. *RoFormer: Enhanced Transformer with Rotary Position Embedding*. 2021. arXiv: 2104.09864. URL: https://arxiv.org/abs/2104.09864.

[13] Hugo Touvron u. a. *LLaMA: Open and Efficient Foundation Language Models*. 2023. arXiv: 2302.13971. URL: https://arxiv.org/abs/2302.13971.

[14] Ashish Vaswani u. a. *Attention Is All You Need*. 2017. arXiv: 1706.03762. URL: https://arxiv.org/abs/1706.03762.